01 / StosONP

ASD1-M1-07 — Stos i obliczanie wyrażeń

Dokumentacja mini-projektu M1

Algorytmy i struktury danych I
Semestr I · rok akademicki 2026/2027
Uniwersytet w Białymstoku, Filia w Wilnie

Autorka: Marzena Magadia
Numer indeksu: 93782
Data przygotowania: 26.10.2026

Przedmiot dokumentacji

W tym projekcie zajęłam się stosem oraz jego wykorzystaniem do sprawdzania nawiasów i obliczania wyrażeń matematycznych. Przygotowana aplikacja pokazuje krok po kroku, co dzieje się na stosie. W dokumentacji opisuję, jak działa program, z jakich technologii korzysta i jak można go przetestować.

Projekt przygotowałam na zajęcia z algorytmów i struktur danych. Termin oddania to 26.10.2026, a obrona odbędzie się 28.10.2026. Podczas obrony mam przedstawić prezentację, pokazać działającą aplikację i odpowiedzieć na pytania.

Status autorstwa i oddania

Do pracy nad projektem włączyłam agenta OpenClaw, któremu przekazałam wymagania zadania. Agent pomagał w przygotowaniu kodu, testów i materiałów. Moim zadaniem było określenie, co ma powstać, sprawdzanie otrzymanych plików i przygotowanie się do wyjaśnienia działania programu.

02 / Cel i zakres

Chciałam przygotować coś więcej niż zwykły kalkulator. Aplikacja ma pokazywać, jak stos zmienia się w kolejnych krokach. Dzięki temu można zobaczyć, kiedy element trafia na stos, kiedy zostaje zdjęty i dlaczego program wykonuje daną operację.

Zrealizowane funkcje

- Samodzielna klasa Stack: push, pop, peek, isEmpty; zabezpieczenie pustego stosu.
- Osobny kontroler trzech rodzajów nawiasów. Pierwszy konflikt wskazuje pozycję; niedomknięcie podajeotwarcia.
- Tokenizacja, sprawdzanie kolejności operandów/operatorów, konwersja stosowa i obliczanie ONP.
- Gotowe scenariusze, własne dane, Krok, Start/Pauza, Reset, trzy tempa.
- Trzy pytania, w tym przewidywanie następnego stanu bieżącego przebiegu.
- Porównanie pięciu wyrażeń, testy automatyczne i prezentacja lokalna.

Kontrakt wejścia

W kalkulatorze można wpisywać liczby od 0 do 99 oraz działania +, −, * i /, a także nawiasy okrągłe. Program nie obsługuje potęg ani liczb ujemnych wpisanych z minusem na początku. Maksymalna długość obliczenia to 60 tokenów. Te ograniczenia przyjęto, żeby przykład był czytelny.

W osobnym trybie można sprawdzić trzy rodzaje nawiasów: (), [] i {}. Program wskazuje pozycję błędu. W polu można wpisać do 500 znaków; pozycje znaków są liczone od zera.

Kryterium poprawności

Na przykład 3 + 4 * (2 − 1) daje wynik 7. Aplikacja pokazuje nie tylko wynik, ale też zapis ONP: 3 4 2 1 - * +. To pomaga zrozumieć kolejność wykonywania działań.

03 / Algorytmy i mechanizmy

Stos i kontrola nawiasów

Podstawą projektu jest stos, czyli struktura działająca według zasady LIFO: ostatni dodany element jest pobierany jako pierwszy. Operacja push dodaje element, pop go usuwa, a peek pozwala podejrzeć ostatni element bez usuwania. Tę samą zasadę wykorzystano przy sprawdzaniu, czy nawiasy są poprawnie zamknięte.

Tokeny i walidacja

Zanim program zacznie liczyć, sprawdza wpisane wyrażenie. Łączy cyfry w liczby i rozpoznaje operatory oraz nawiasy. Odrzuca np. 2++3, 2 3 i 2(3), ponieważ taki zapis nie pasuje do przyjętych zasad.

Konwersja stosowa — Shunting Yard

Do zmiany zwykłego zapisu działań na ONP wykorzystano algorytm Shunting Yard. Liczby są dopisywane do wyniku, a operatory tymczasowo trafiają na stos. O ich kolejności decyduje priorytet działań i nawiasy. Dlatego w 2+3*4 mnożenie zostanie wykonane przed dodawaniem.

Obliczanie ONP

Gdy wyrażenie jest już zapisane w ONP, program odczytuje je od lewej do prawej. Liczby odkłada na stos. Po napotkaniu operatora pobiera dwie ostatnie liczby, wykonuje działanie i odkłada wynik. Dodatkowo sprawdza błędy, np. dzielenie przez zero.

Ważna obserwacja: stos przechowuje wyniki częściowych obliczeń, które będą potrzebne w kolejnych krokach.

04 / Struktury danych i złożoność

W projekcie stos został zrealizowany jako osobna klasa Stack. Dane są przechowywane w obiekcie, a licznik size określa liczbę elementów. Dzięki temu mogę pokazać działanie push, pop i peek bez korzystania z gotowych metod stosu.

Metoda	Czas algorytmu	Pamięć pomocnicza
push / pop / peek	O(1)	O(1) na operację
Kontrola nawiasów	O(m)	O(m)
Konwersja ONP	O(n)	O(n)
Obliczanie ONP	O(n)	O(n)
Tokenizacja + walidacja	O(m+n)	O(n)

Tabela 1. Koszt rdzenia w modelu stałego kosztu operacji liczbowej i dostępu indeksowego; m — znaki tekstu, n — tokeny.

Każdy operator trafia na stos tylko raz i jest z niego zdejmowany najwyżej raz. Dlatego konwersja wyrażenia do ONP ma złożoność O(n), gdzie n oznacza liczbę tokenów. Przy sprawdzaniu nawiasów potrzebna pamięć zależy od liczby otwartych nawiasów.

Liczniki

W programie zliczane są operacje push (P), pop (Z), peek (O) i największa wysokość stosu (H). Pozwala to porównać różne wyrażenia, a nie tylko ich wyniki.

Rzeczywisty koszt wizualizacji

Aplikacja zapisuje też kolejne stany stosu, żeby można je było odtwarzać. To wymaga dodatkowej pamięci i czasu.
Dlatego złożoność samego algorytmu nie jest tym samym co koszt wyświetlania animacji w przeglądarce.

05 / Technologie i uruchomienie

Dlaczego HTML, CSS i JavaScript?

Projekt działa jako strona internetowa. HTML odpowiada za zawartość i przyciski, CSS za wygląd, a JavaScript za działanie algorytmów i animację. Wybrałam tę formę, ponieważ aplikację można uruchomić lokalnie, otwierając plik index.html w przeglądarce, bez zakładania konta i bez serwera.

Uruchomienie produktu

1. Skopiuj cały katalog projektu, zachowując układ plików.
2. Otwórz index.html w aktualnej przeglądarce z JavaScript.
3. Wybierz tryb, wpisz dane lub wybierz scenariusz.
4. Uruchom testy przyciskiem w sekcji Testy.
5. Otwórz presentation.html jako osobną prezentację; strzałki zmieniają slajdy.

Uruchomienie testów bez interfejsu

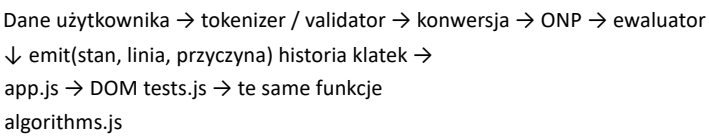
Aby sprawdzić program, wystarczy otworzyć cały folder projektu, a następnie uruchomić plik index.html. Można wybrać gotowy przykład albo wpisać własne wyrażenie, przechodzić kolejne kroki oraz uruchomić testy.
Prezentacja znajduje się w osobnym pliku presentation.html.

Narzędzia weryfikacji

Testy algorytmów można również uruchomić w Node.js poleceniem node tests.js. Wyniki zapisują się w pliku test-results.json. Dodatkowe porównania wykonuje skrypt comparison.cjs.

W przygotowanych materiałach zapisano także testy w przeglądarce Edge z użyciem Playwright Core. Narzędzie to służyło do sprawdzania interfejsu i nie jest potrzebne osobie, która chce po prostu otworzyć aplikację.

06 / Architektura i realizacja



Rys. 1. Przepływ danych w realizacji StosONP.

Odpowiedzialności plików

Najważniejsza logika programu znajduje się w algorithms.js. Plik app.js odpowiada za obsługę przycisków, animacji i pytań. index.html oraz style.css tworzą wygląd strony, a tests.js zawiera testy. Dzięki takiemu podziałowi łatwiej odnaleźć potrzebną część projektu.

Rzeczywiste kroki

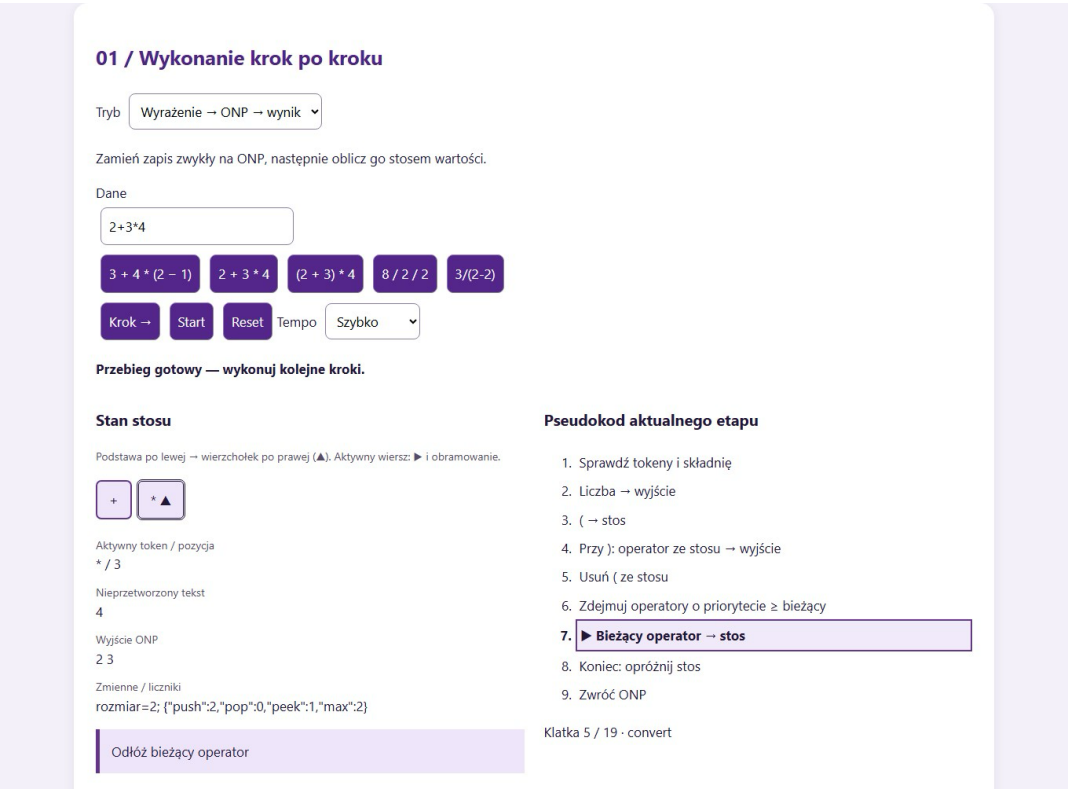
Po wpisaniu wyrażenia program przygotowuje kolejne stany obliczenia. Przycisk „Krok” pokazuje jeden następny stan, a „Start” odtwarza je automatycznie. Mogę więc zatrzymać pokaz i wyjaśnić, co dokładnie wydarzyło się w danym momencie.

Etapy wykonania

1. Implementacja własnego stosu oraz walidatora.
2. Konwersja, ewaluator i kontroler nawiasów z licznikami.
3. Testy wyników, błędów i stanów pośrednich.
4. Interfejs, pytania, porównania i rzeczywisty test przeglądarkowy.
5. Osobne slajdy, raport, instrukcja obrony.

W projekcie przygotowano kolejno: klasę stosu, sprawdzanie nawiasów, konwersję do ONP, obliczanie wyniku, testy oraz interfejs. Na końcu powstały materiały do prezentacji i dokumentacja.

07 / Interfejs i obsługa



Rys. 2. Rzeczywisty zrzut Edge: wyrażenie 2+3*4, stos operatorów po odłożeniu *. Źródło: screenshot-app.png wygenerowany przez test UI.

Na ekranie widoczny jest aktualny stan stosu i zaznaczony krok algorytmu. Symbol ▲ oznacza wierzchołek stosu, a Ø informuje, że stos jest pusty. To pozwala obserwować działanie programu bez zaglądania do kodu.

Przycisk „Krok” przesuwa pokaz o jeden etap. „Start/Pauza” uruchamia i zatrzymuje animację, a „Reset” wraca do początku. Można też zmienić tempo, aby spokojnie prześledzić trudniejsze fragmenty.

W trybie ręcznym można dodawać i usuwać elementy stosu oraz podejrzeć jego wierzchołek. Program nie pozwala pobrać elementu z pustego stosu. Są też pytania, w których trzeba przewidzieć następny stan.

08 / Porównanie i interpretacja

Porównałam przykłady przygotowane w projekcie. Program zapisuje liczbę operacji na stosie i największą liczbę przechowywanych elementów. Szczegółowe wyniki znajdują się w pliku comparison-results.json.

Wejście / n	Wynik	Konwersja P/Z/O/H	Ewaluacja P/Z/O/H
2+3*4 / 5	14	2/2/1/2	5/4/1/3
(2+3)*4 / 7	20	3/3/3/2	5/4/1/2
12+3*(8-2)/2 / 11	21	5/5/6/4	9/8/1/4
(15-3)*(2+4)/3 / 13	24	6/6/7/3	9/8/1/3
99/(3+6)+7*(5-2) / 15	32	7/7/8/4	11/10/1/4

Tabela 2. Rzeczywiste liczniki stosu; definicje na stronie 4.

Kolejność operatorów

Najprościej zauważyć różnicę na przykładzie 2+3*4 oraz (2+3)*4. Pierwsze wyrażenie daje 14, ponieważ mnożenie ma pierwszeństwo. Drugie daje 20, ponieważ nawias każe najpierw dodać 2 i 3. Inna jest też kolejność operatorów w ONP.

Przy pierwszym przykładzie konwersja wykonuje łącznie 5 operacji push, pop i peek, a przy drugim 9. W obu przypadkach obliczanie ONP daje łącznie 10 takich operacji. To pokazuje, że nawiasy mogą zmieniać pracę stosu.

Wnioski z dłuższych danych

Z porównania dłuższych przykładów wynika, że większa liczba tokenów nie musi oznaczać wyższego stosu. Znaczenie ma także układ działań i nawiasów. Nie mierzyłam czasu działania programu — porównanie dotyczy liczników operacji.

09 / Testowanie i wyniki

W materiałach projektu zapisano 36 zaliczonych testów algorytmicznych oraz 19 zaliczonych kontroli przeglądarkowych. Testy sprawdzały między innymi działanie stosu, nawiasy, wyniki działań, dzielenie przez zero i przyciski aplikacji. Wyniki są zapisane w plikach test-results.json i browser-results.json. Przed oddaniem warto uruchomić je ponownie.

Przykład / etap	Oczekiwano i uzyskano	Status
push 2,7; peek, pop, pop	7,7,2; pusty	PASS
pop/peek pustego	oba odrzucone	PASS
([)] / kontroler	błąd @2	PASS
a([/ kontroler	otwarcia @1,@2	PASS
3+4*(2−1)	3 4 2 1 - * +; 7	PASS
8/2/2; 99; 3/2	2; 99; 1.5	PASS
2+3*4 po odłożeniu *	stos [+,*]	PASS
2 3 4 * + po *	stos [2,12]	PASS
2 7 -	-5	PASS
puste, 2++3, -2, 2 3	odrzucenie każdego	PASS
3/(2-2); ONP 1 +	zero; za mało operandów	PASS

Tabela 3. Wybrane rzeczywiste testy; zbiorcze wiersze odpowiadają osobnym przypadkom w tests.js.

Kontrole interfejsu obejmowały m.in. przyciski „Krok”, „Reset” i „Pauza”, zmianę danych, pytania oraz komunikaty o błędach. Nie oznacza to jednak, że program został przetestowany we wszystkich przeglądarkach.

Podczas sprawdzania poprawiono kilka szczegółów związanych z wyświetlaniem danych i obsługą błędów. Dzięki temu zapisane kroki odpowiadają faktycznemu przebiegowi obliczenia.

10 / Podsumowanie, źródła i obrona

Wnioski i rozwój

Praca nad tym projektem pomogła mi lepiej zrozumieć, do czego służy stos. Wcześniej znałam przede wszystkim wynik działania, a tutaj mogę zobaczyć cały proces: odkładanie operatorów, zdejmowanie elementów i obliczanie wyrażenia w ONP. W przyszłości można byłoby rozbudować aplikację o kolejne działania i bardziej złożone liczby.

Narzędzia AI i rzeczywisty wkład

Do wykonania projektu dołączyłam agenta OpenClaw. Uruchomiłam środowisko Windows 11 w UTM na MacBooku, a następnie korzystałam z OpenClaw i połączonego modelu GPT-6 Astra. Przekazałam agentowi treść zadania i wymagania dotyczące aplikacji, testów, prezentacji oraz dokumentacji. Agent przygotowywał pliki i pomagał w ich poprawianiu. Sprawdzałam, jakie pliki powstały, otwierałam materiały i zgłaszałam potrzebne zmiany.

Źródła

1. Karta ASD1-M1-07 przekazana w Dzienniku UwB: zakres, przykłady, ograniczenia i kryteria; dostarczona 13.10.2026.
2. Princeton, Algorithms — Bags, Queues, and Stacks: https://algs4.cs.princeton.edu/13stacks/ — materiał donauki wskazanego tematu; nie kopiowano kodu.
3. Emory University — Shunting Yard, materiał wymieniony w karcie. Dokładny odnośnik należy przepisać zDziennika; nie weryfikowano konkretnej strony internetowej podczas realizacji.